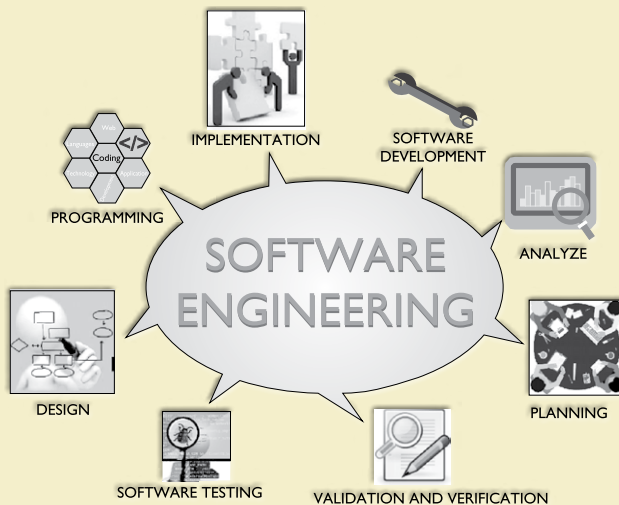


CHAPTER

6

FUNCTION-ORIENTED SOFTWARE DESIGN



CHAPTER OUTLINE

- ◆ Overview of SA/SD Methodology
- ◆ Structured Analysis
- ◆ Developing the DFD Model of a System
- ◆ Structured Design
- ◆ Detailed Design
- ◆ Design Review

The belief that complex systems require armies of designers and programmers is wrong. A system that is not understood in its entirety, or at least to a significant degree of detail by a single individual, should probably not be built.

—Niklaus Wirth

LEARNING OBJECTIVES

- ◆ Basic concepts in structured analysis and structured design
- ◆ Development of the DFD model of a system
- ◆ Transformation of DFD model into a structure chart model

Function-oriented design techniques were proposed nearly four decades ago. These techniques are at the present time still very popular and are currently being used in many software development projects. These techniques, to start with, view a system as a black-box that provides a set of services to the users of the software. These services provided by a software (e.g., issue book, search book, etc., for a Library Automation Software to its users are also known as the high-level functions supported by the software. During the design process, these high-level functions are successively decomposed into more detailed functions.

The term *top-down decomposition* is often used to denote the successive decomposition of a set of high-level functions into more detailed functions.

After top-down decomposition has been carried out, the different identified functions are mapped to modules and a module structure is created. This module structure would possess all the characteristics of a good design identified in the last chapter.

In this text, we shall not focus on any specific design methodology. Instead, we shall discuss a methodology that has the essential features of several important function-oriented design methodologies. Such an approach shall enable us to easily assimilate any specific design methodology in the future whenever the need arises. Learning a specific methodology may become necessary for you later, since different software development houses follow different methodologies. After all, the different procedural design techniques can be considered as sister techniques that have only minor differences with respect to the methodology and notations. We shall call the design technique discussed in this text as *structured analysis/structured design* (SA/SD) methodology. This technique draws heavily from the design methodologies proposed by the following authors:

- DeMarco and Yourdon [1978]
- Constantine and Yourdon [1979]
- Gane and Sarson [1979]
- Hatley and Pirbhai [1987]

The SA/SD technique can be used to perform the high-level design of a software. The details of SA/SD technique are discussed further.

6.1 OVERVIEW OF SA/SD METHODOLOGY

As the name itself implies, SA/SD methodology involves carrying out two distinct activities:

- Structured analysis (SA)
- Structured design (SD)

The roles of structured analysis (SA) and structured design (SD) have been shown schematically in Figure 6.1. Observe the following from the figure:

- During structured analysis, the SRS document is transformed into a *data flow diagram* (DFD) model.
- During structured design, the DFD model is transformed into a structure chart.

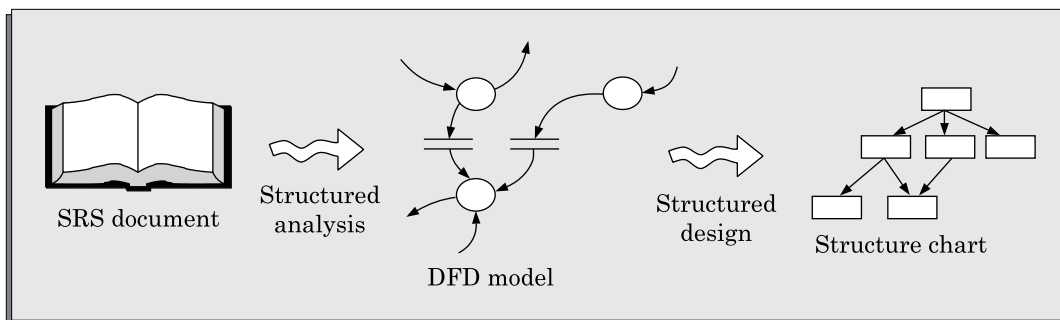


FIGURE 6.1 Structured analysis and structured design methodology.

As shown in Figure 6.1, the structured analysis activity transforms the SRS document into a graphic model called the DFD model. During structured analysis, functional decomposition of the system is achieved. That is, each function that the system needs to perform is analysed and hierarchically decomposed into more detailed functions. On the other hand, during structured design, all functions identified during structured analysis are mapped to a module structure. This module structure is also called the *high-level design* or the *software architecture* for the given problem. This is represented using a structure chart.

The high-level design stage is normally followed by a detailed design stage. During the detailed design stage, the algorithms and data structures for the individual modules are designed. The detailed design can directly be implemented as a working system using a conventional programming language.

The results of structured analysis can therefore, be easily understood by the user. In fact, the different functions and data in structured analysis are named using the user's terminology. The user can therefore even review the results of the structured analysis to ensure that it captures all his requirements.

In the following section, we first discuss how to carry out structured analysis to construct the DFD model. Subsequently, we discuss how the DFD model can be transformed into structured design.

It is important to understand that the purpose of structured analysis is to capture the detailed structure of the system as perceived by the user, whereas the purpose of structured design is to define the structure of the solution that is suitable for implementation in some programming language.

6.2 STRUCTURED ANALYSIS

We have already mentioned that during structured analysis, the major processing tasks (high-level functions) of the system are analysed, and the data flow among these processing tasks are represented graphically. Significant contributions to the development of the structured analysis techniques have been made by Gane and Sarson [1979], and DeMarco and Yourdon [1978]. The structured analysis technique is based on the following underlying principles:

- Top-down decomposition approach.
- Application of divide and conquer principle. Through this each high-level function is independently decomposed into detailed functions.
- Graphical representation of the analysis results using *data flow diagrams* (DFDs).

DFD representation of a problem, as we shall see shortly, is very easy to construct. Though extremely simple, it is a very powerful tool to tackle the complexity of industry standard problems.

Please note that a DFD model only represents the data flow aspects and does not show the sequence of execution of the different functions and the conditions based on which a function may or may not be executed. In fact, it completely ignores aspects such as control flow, the specific algorithms used by the functions, etc. In the DFD terminology, each function is called a *process* or a *bubble*. It is useful to consider each function as a processing station (or process) that consumes some input data and produces some output data.

DFD is an elegant modelling technique that can be used not only to represent the results of structured analysis of a software problem, but also useful for several other applications such as showing the flow of documents or items in an organisation. Recall that in Chapter 1 we had given an example (see Figure 1.10) to illustrate how a DFD can be used to represent the processing activities and flow of material in an automated car assembling plant. We now elaborate how a DFD model can be constructed.

A DFD is a hierarchical graphical model of a system that shows the different processing activities or functions that the system performs and the data interchange among those functions.

6.2.1 Data Flow Diagrams (DFDs)

The DFD (also known as the *bubble chart*) is a simple graphical formalism that can be used to represent a system in terms of the input data to the system, various processing carried out on those data, and the output data generated by the system. The main reason why the DFD technique is so popular is probably because of the fact that DFD is a very simple formalism—it is simple to understand and use. A DFD model uses a very limited number of primitive symbols (shown in Figure 6.2) to represent the functions performed by a system and the data flow among these functions.

Starting with a set of high-level functions that a system performs, a DFD model represents the subfunctions performed by the functions using a hierarchy of diagrams. We had pointed out while discussing the principle of abstraction in Section 1.3.2 that any hierarchical representation is an effective means to tackle complexity. Human mind is such

that it can easily understand any hierarchical model of a system—because in a hierarchical model, starting with a very abstract model of a system, various details of the system are slowly introduced through different levels of the hierarchy. The DFD technique is also based on a very simple set of intuitive concepts and rules. We now elaborate the different concepts associated with building a DFD model of a system.

Primitive symbols used for constructing DFDs

There are essentially five different types of symbols used for constructing DFDs. These primitive symbols are depicted in Figure 6.2. The meaning of these symbols are explained as follows:

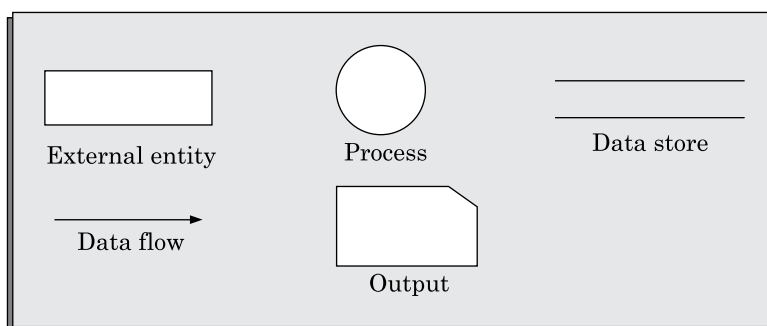


FIGURE 6.2 Symbols used for designing DFDs.

Function symbol: A function is represented using a circle. This symbol is called a *process* or a *bubble*. Bubbles are annotated with the names of the corresponding functions (see Figure 6.3).

External entity symbol: An external entity such as a librarian, a library member, etc. is represented by a rectangle. The external entities are essentially those physical entities external to the software system which interact with the system by inputting data to the system or by consuming the data produced by the system. In addition to the human users, the external entity symbols can be used to represent external hardware and software such as another application software that would interact with the software being modelled.

Data flow symbol: A directed arc (or an arrow) is used as a data flow symbol. A data flow symbol represents the data flow occurring between two processes or between an external entity and a process in the direction of the data flow arrow. Data flow symbols are usually annotated with the corresponding data names. For example the DFD in Figure 6.3(a) shows three data flows—the data item *number* flowing from the process *read-number* to *validate-number*, data-item *flowing into* *read-number*, and *valid-number* flowing out of *validate-number*.

Data store symbol: A data store is represented using two parallel lines. It represents a logical file. That is, a data store symbol can represent either a data structure or a physical file on disk. Each data store is connected to a process by means of a data flow symbol. The direction of the data flow arrow shows whether data is being read from or written into a data store. An arrow flowing in or out of a data store implicitly represents the entire

data of the data store and hence arrows connecting to a data store need not be annotated with the name of the corresponding data items. As an example of a data store, *number* is a data store in Figure 6.3(b).

Output symbol: The output symbol is as shown in Figure 6.2. The output symbol is used when a hard copy is produced.

The notations that we are following in this text are closer to the Yourdon's notations than to the other notations. You may sometimes find notations in other books that are slightly different than those discussed here. For example, the data store may look like a box with one end open. That is because, they may be following notations such as those of Gane and Sarson [1979].

Important concepts associated with constructing DFD models

Before we discuss how to construct the DFD model of a system, let us discuss some important concepts associated with DFDs:

Synchronous and asynchronous operations

If two bubbles are directly connected by a data flow arrow, then they are synchronous. This means that they operate at the same speed. An example of such an arrangement is shown in Figure 6.3(a). Here, the *validate-number* bubble can start processing only after the *read-number* bubble has supplied data to it; and the *read-number* bubble has to wait until the *validate-number* bubble has consumed its data.

However, if two bubbles are connected through a data store, as in Figure 6.3(b) then the speed of operation of the bubbles are independent. This statement can be explained using the following reasoning. The data produced by a producer bubble gets stored in the data store. It is therefore possible that the producer bubble stores several pieces of data items, even before the consumer bubble consumes any of them.

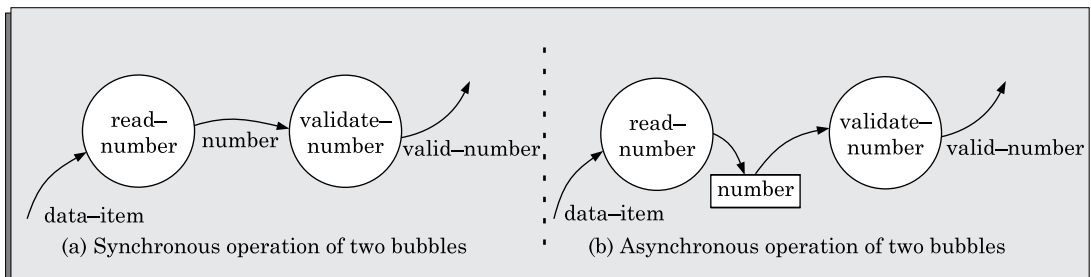


FIGURE 6.3 Synchronous and asynchronous data flow.

Data dictionary

Every DFD model of a system must be accompanied by a data dictionary. A data dictionary lists all data items that appear in a DFD model. The data items listed include all data flows and the contents of all data stores appearing on all the DFDs in a DFD model. Please remember that

A data dictionary lists the purpose of all data items and the definition of all composite data items in terms of their component data items.

the DFD model of a system typically consists of several DFDs, viz., level 0 DFD, level 1 DFD, level 2 DFDs, etc., as shown in Figure 6.4 discussed in new subsection. However, a single data dictionary should capture all the data appearing in all the DFDs constituting the DFD model of a system.

For example, a data dictionary entry may represent that the data *grossPay* consists of the components *regularPay* and *overtimePay*.

$$\text{grossPay} = \text{regularPay} + \text{overtimePay}$$

For the smallest units of data items, the data dictionary simply lists their name and their type. Composite data items are expressed in terms of the component data items using certain operators. The operators using which a composite data item can be expressed in terms of its component data items are discussed subsequently.

The dictionary plays a very important role in any software development process, especially for the following reasons:

- A data dictionary provides a standard terminology for all relevant data for use by the developers working in a project. A consistent vocabulary for data items is very important, since in large projects different developers of the project have a tendency to use different terms to refer to the same data, which unnecessarily causes confusion.
- The data dictionary helps the developers to determine the definition of different data structures in terms of their component elements while implementing the design.
- The data dictionary helps to perform impact analysis. That is, it is possible to determine the effect of some data on various processing activities and *vice versa*. Such impact analysis is especially useful when one wants to check the impact of changing an input value type, or a bug in some functionality, etc.

For large systems, the data dictionary can become extremely complex and voluminous. Even moderate-sized projects can have thousands of entries in the data dictionary. It becomes extremely difficult to maintain a voluminous dictionary manually. *Computer-aided software engineering* (CASE) tools come handy to overcome this problem. Most CASE tools usually capture the data items appearing in a DFD as the DFD is drawn, and automatically generate the data dictionary. As a result, the designers do not have to spend almost any effort in creating the data dictionary. These CASE tools also support some query language facility to query about the definition and usage of data items. For example, queries may be formulated to determine which data item affects which processes, or a process affects which data items, or the definition and usage of specific data items, etc. Query handling is facilitated by storing the data dictionary in a *relational database management system* (RDBMS).

Data definition

Composite data items can be defined in terms of primitive data items using the following data definition operators.

- + : denotes composition of two data items, e.g. $a+b$ represents data a and b .
- [, ,] : represents selection, i.e. any one of the data items listed inside the square bracket can occur. For example, $[a, b]$ represents either a occurs or b occurs.
- () : the contents inside the bracket represent optional data which may or may not appear.

- $a+(b)$ represents either a or $a+b$ occurs.
- $\{\}$: represents iterative data definition, e.g. $\{\text{name}\}5$ represents five `name` data. $\{\text{name}\}^*$ represents zero or more instances of `name` data.
- $=$: represents equivalence, e.g. $a=b+c$ means that a is a composite data item comprising of both b and c .
- $/**/$: Anything appearing within $/*$ and $*/$ is considered as comment.

6.3 DEVELOPING THE DFD MODEL OF A SYSTEM

A DFD model of a system graphically represents how each input data is transformed to its corresponding output data through a hierarchy of DFDs.

The DFD model of a system is constructed by using a hierarchy of DFDs (see [Figure 6.4](#)). The top level DFD is called the level 0 DFD or the context diagram. This is the most abstract (simplest) representation of the system (highest level). It is the easiest to draw and understand.

The DFD model of a problem consists of many DFDs and a single data dictionary.

At each successive lower level DFDs, more and more details are gradually introduced. To develop a higher-level DFD model, processes are decomposed into their subprocesses and the data flow among these subprocesses are identified.

To develop the data flow model of a system, first the most abstract representation (highest level) of the problem is to be worked out. Subsequently, the lower level DFDs are developed. Level 0 and Level 1 consist of only one DFD each. Level 2 may contain up to 7 separate DFDs, and level 3 up to 49 DFDs, and so on. However, there is only a single data dictionary for the entire DFD model. All the data names appearing in all DFDs are populated in the data dictionary and the data dictionary contains the definitions of all the data items.

6.3.1 Context Diagram

The context diagram is the most abstract (highest level) data flow representation of a system. It represents the entire system as a single bubble. The bubble in the context diagram is annotated with the name of the software system being developed (usually a noun). This is the only bubble in a DFD model, where a noun is used for naming the bubble. The bubbles at all other levels are annotated with verbs according to the main function performed by the bubble. This is expected since the purpose of the context diagram is to capture the context of the system rather than its functionality. As an example of a context diagram, consider the context diagram a software developed to automate the book keeping activities of a supermarket (see [Figure 6.10](#)). The context diagram has been labelled as ‘Supermarket software’.

The context diagram establishes the context in which the system operates; that is, who are the users, what data do they input to the system, and what data they receive from the system.

The name context diagram of the level 0 DFD is justified because it represents the context in which the system would exist; that is, the external entities who would interact with the system and the specific data items that they would be supplying the system and

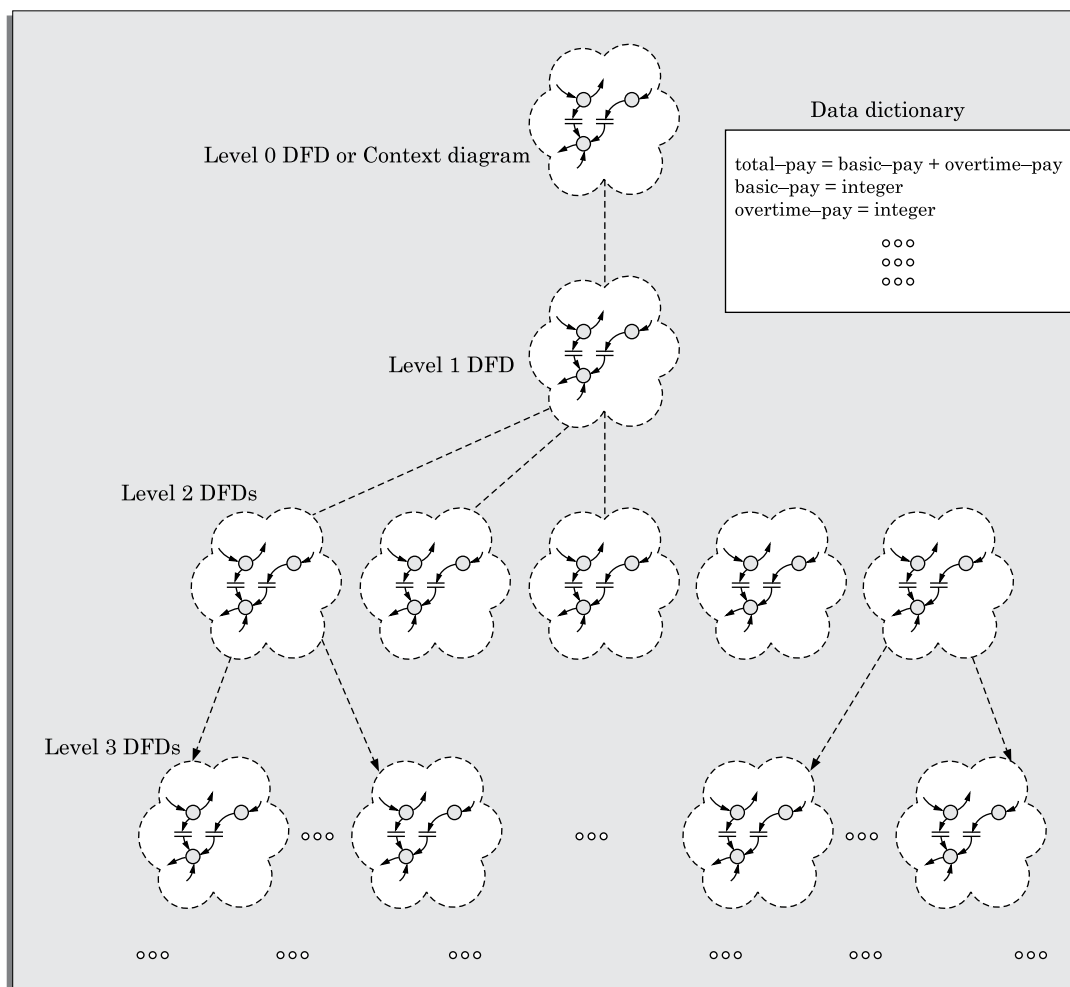


FIGURE 6.4 DFD model of a system consists of a hierarchy of DFDs and a single data dictionary.

the data items they would be receiving from the system. The various external entities with which the system interacts and the data flow occurring between the system and the external entities are represented. The data input to the system and the data output from the system are represented as incoming and outgoing arrows. These data flow arrows should be annotated with the corresponding data names.

To develop the context diagram of the system, we have to analyse the SRS document to identify the different types of users who would be using the system and the kinds of data they would be inputting to the system and the data they would be receiving from the system. Here, the term users of the system also includes any external systems which supply data to or receive data from the system.

6.3.2 Level 1 DFD

The level 1 DFD usually contains three to seven bubbles. That is, the system is represented as performing three to seven important functions. To develop the level 1 DFD, examine the high-level functional requirements in the SRS document. If there are three to seven high-level functional requirements, then each of these can be directly represented as a bubble in the level 1 DFD. Next, examine the input data to these functions and the data output by these functions as documented in the SRS document and represent them appropriately in the diagram.

What if a system has more than seven high-level requirements identified in the SRS document? In this case, some of the related requirements have to be combined and represented as a single bubble in the level 1 DFD. These can be split appropriately in the lower DFD levels. If a system has less than three high-level functional requirements, then some of the high-level requirements need to be split into their subfunctions so that we have roughly about five to seven bubbles represented on the diagram. We illustrate construction of level 1 DFDs in Examples 6.1 to 6.4.

Decomposition

Each bubble in the DFD represents a function performed by the system. The bubbles are decomposed into subfunctions at the successive levels of the DFD model. Decomposition of a bubble is also known as *factoring* or *exploding* a bubble. Each bubble at any level of DFD is usually decomposed to anything three to seven bubbles. A few bubbles at any level make that level superfluous. For example, if a bubble is decomposed to just one bubble or two bubbles, then this decomposition becomes trivial and redundant. On the other hand, too many bubbles (i.e. more than seven bubbles) at any level of a DFD makes the DFD model hard to understand. Decomposition of a bubble should be carried on until a level is reached at which the function of the bubble can be described using a simple algorithm.

We can now describe how to go about developing the DFD model of a system more systematically.

1. **Construction of context diagram:** Examine the SRS document to determine:
 - Different high-level functions that the system needs to perform.
 - Data input to every high-level function.
 - Data output from every high-level function.
 - Interactions (data flow) among the identified high-level functions.

Represent these aspects of the high-level functions in a diagrammatic form. This would form the top-level *data flow diagram* (DFD), usually called the DFD 0.

2. **Construction of level 1 diagram:** Examine the high-level functions described in the SRS document. If there are three to seven high-level requirements in the SRS document, then represent each of the high-level function in the form of a bubble. If there are more than seven bubbles, then some of them have to be combined. If there are less than three bubbles, then some of these have to be split.
3. **Construction of lower-level diagrams:** Decompose each high-level function into its constituent subfunctions through the following set of activities:

- Identify the different subfunctions of the high-level function.
- Identify the data input to each of these subfunctions.
- Identify the data output from each of these subfunctions.
- Identify the interactions (data flow) among these subfunctions.

Represent these aspects in a diagrammatic form using a DFD.

Recursively repeat Step 3 for each subfunction until a subfunction can be represented by using a simple algorithm.

Numbering of bubbles

It is necessary to number the different bubbles occurring in the DFD. These numbers help in uniquely identifying any bubble in the DFD from its bubble number. The bubble at the context level is usually assigned the number 0 to indicate that it is the 0 level DFD. Bubbles at level 1 are numbered, 0.1, 0.2, 0.3, etc. When a bubble numbered x is decomposed, its children bubble are numbered $x.1$, $x.2$, $x.3$, etc. In this numbering scheme, by looking at the number of a bubble we can unambiguously determine its level, its ancestors, and its successors.

Balancing DFDs

The DFD model of a system usually consists of many DFDs that are organised in a hierarchy. In this context, a DFD is required to be balanced with respect to the corresponding bubble of the parent DFD.

We illustrate the concept of balancing a DFD in Figure 6.5. In the level 1 DFD, data items $d1$ and $d3$ flow out of the bubble 0.1 and the data item $d2$ flows into the bubble 0.1 (shown by the dotted circle). In the next level, bubble 0.1 is decomposed into three DFDs (0.1.1, 0.1.2, 0.1.3). The decomposition is balanced, as $d1$ and $d3$ flow out of the level 2 diagram and $d2$ flows in. Please note that dangling arrows ($d1$, $d2$, $d3$) represent the data flows into or out of a diagram.

The data that flow into or out of a bubble must match the data flow at the next level of DFD. This is known as *balancing* a DFD.

How far to decompose?

A bubble should not be decomposed any further once a bubble is found to represent a simple set of instructions. For simple problems, decomposition up to level 1 should suffice. However, large industry standard problems may need decomposition up to level 3 or level 4. Rarely, if ever, decomposition beyond level 4 is needed.

Commonly made errors while constructing a DFD model

Although DFDs are simple to understand and draw, students and practitioners alike encounter similar types of problems while modelling software problems using DFDs. While learning from experience is a powerful thing, it is an expensive pedagogical technique in the business world. It is therefore useful to understand the different types of mistakes that beginners usually make while constructing the DFD model of systems, so that you can consciously try to avoid them. The errors are as follows:

- Many beginners commit the mistake of drawing more than one bubble in the context diagram. Context diagram should depict the system as a single bubble.

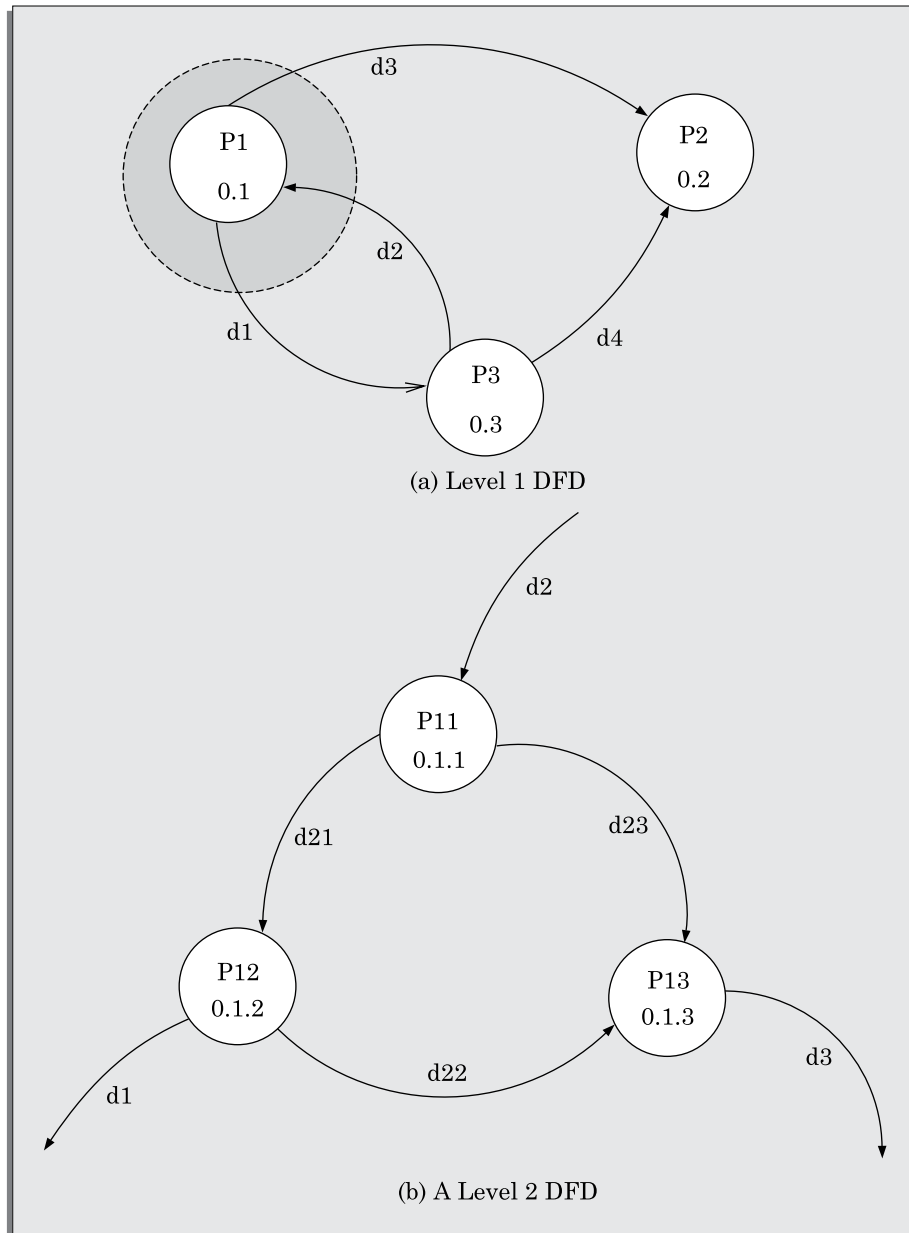


FIGURE 6.5 An example showing balanced decomposition.

- Many beginners create DFD models in which external entities appearing at all levels of DFDs. All external entities interacting with the system should be represented only in the context diagram. The external entities should not appear in the DFDs at any other level.

- It is a common oversight to have either too few or too many bubbles in a DFD. Only three to seven bubbles per diagram should be allowed. This also means that each bubble in a DFD should be decomposed three to seven bubbles in the next level.
- Many beginners leave the DFDs at the different levels of a DFD model unbalanced.
- A common mistake committed by many beginners while developing a DFD model is attempting to represent control information in a DFD.

It is important to realise that a DFD represents only data flow, and it does not represent any control information.

The following are some illustrative mistakes of trying to represent control aspects such as:

ILLUSTRATION 1 A book can be searched in the library catalog by inputting its name. If the book is available in the library, then the details of the book are displayed. If the book is not listed in the catalog, then an error message is generated. While developing the DFD model for this simple problem, many beginners commit the mistake of drawing an arrow (as shown in Figure 6.6) to indicate that the error function is invoked after the search book. But, this is a control information and should not be shown on the DFD.

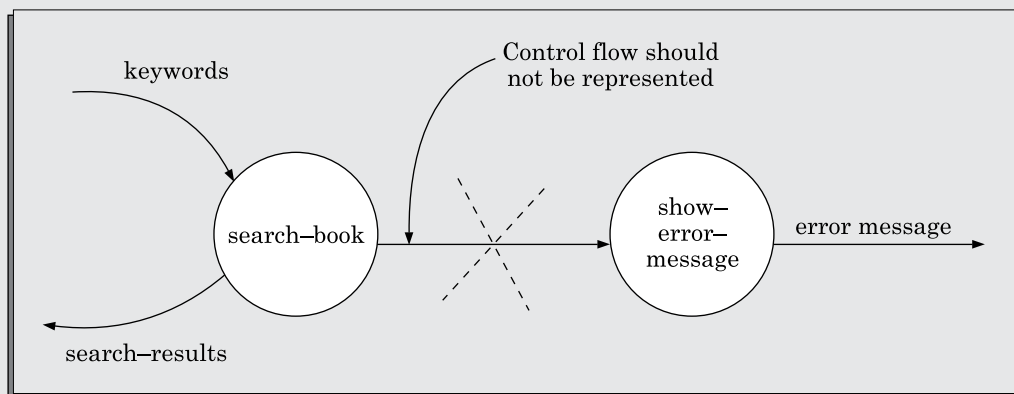


FIGURE 6.6 It is incorrect to show control information on a DFD.

ILLUSTRATION 2 Another type of error occurs when one tries to represent when or in what order different functions (processes) are invoked. A DFD similarly should not represent the conditions under which different functions are invoked.

ILLUSTRATION 3 If a bubble A invokes either the bubble B or the bubble C depending upon some conditions, we need only to represent the data that flows between bubbles A and B or bubbles A and C and not the conditions depending on which the two modules are invoked.

- A data flow arrow should not connect two data stores or even a data store with an external entity. Thus, data cannot flow from a data store to another data store or to an external entity without any intervening processing. As a result, a data store should be connected only to bubbles through data flow arrows.

- All the functionalities of the system must be captured by the DFD model. No function of the system specified in the SRS document of the system should be overlooked.
- Only those functions of the system specified in the SRS document should be represented. That is, the designer should not assume functionality of the system not specified by the SRS document and then try to represent them in the DFD.
- Incomplete data dictionary and data dictionary showing incorrect composition of data items are other frequently committed mistakes.
- The data and function names must be intuitive. Some students and even practicing developers use meaningless symbolic data names such as a,b,c, etc. Such names hinder understanding the DFD model.
- Novices usually clutter their DFDs with too many data flow arrow. It becomes difficult to understand a DFD if any bubble is associated with more than seven data flows. When there are too many data flowing in or out of a DFD, it is better to combine these data items into a high-level data item. Figure 6.7 shows an example concerning how a DFD can be simplified by combining several data flows into a single high-level data flow.

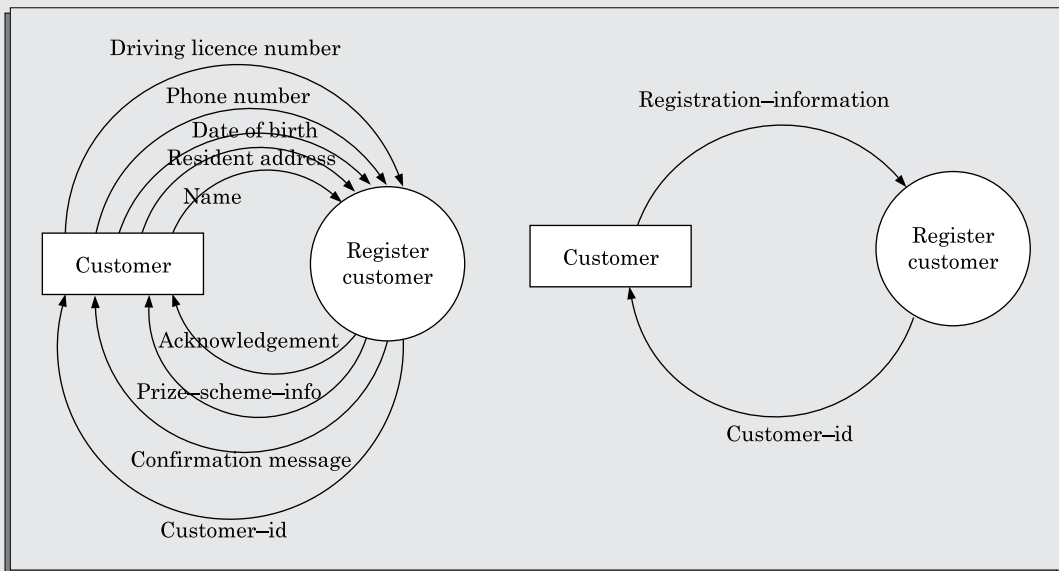


FIGURE 6.7 Illustration of how to avoid data cluttering.

We now illustrate the structured analysis technique through a few examples.

EXAMPLE 6.1 (RMS Calculating Software) A software system called RMS calculating software would read three integral numbers from the user in the range of -1000 and $+1000$ and would determine the *root mean square* (RMS) of the three input numbers and display it.

In this example, the context diagram is simple to draw. The system accepts three integers from the user and returns the result to him. This has been shown in Figure 6.8(a). To draw the level 1 DFD, from a cursory analysis of the problem description, we can see that there are four basic functions that the system needs to perform—accept the input numbers from the user, validate the numbers, calculate the root mean square of the input numbers and, then display the result. After representing these four functions in Figure 6.8(b), we observe that the calculation of root mean square essentially consists of the functions—calculate the squares of the input numbers, calculate the mean, and finally calculate the root. This decomposition is shown in the level 2 DFD in Figure 6.8(c).

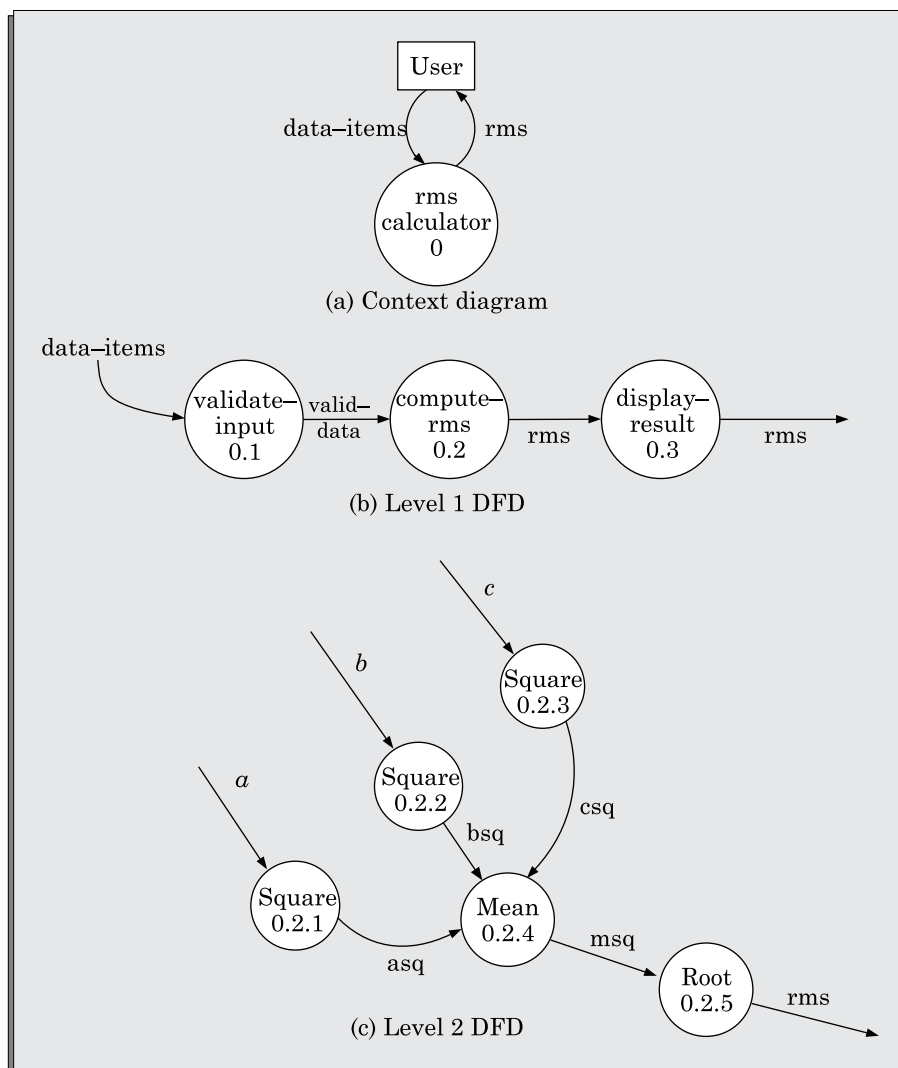


FIGURE 6.8 Context diagram, level 1, and level 2 DFDs for Example 6.1.

Data dictionary for the DFD model of Example 6.1

```
data-items: {integer}3
rms: float
valid-data:data-items
a: integer
b: integer
c: integer
asq: integer
bsq: integer
csq: integer
msq: integer
```

Example 6.1 is an almost trivial example and is only meant to illustrate the basic methodology. Now, let us perform the structured analysis for a more complex problem.

EXAMPLE 6.2 (Tic-Tac-Toe Computer Game) Tic-tac-toe is a computer game in which a human player and the computer make alternate moves on a 3×3 square. A move consists of marking a previously unmarked square. The player who is first to place three consecutive marks along a straight line (i.e., along a row, column, or diagonal) on the square wins. As soon as either of the human player or the computer wins, a message congratulating the winner should be displayed. If neither player manages to get three consecutive marks along a straight line and all the squares on the board are filled up, then the game is drawn. The computer always tries to win a game.

The context diagram and the level 1 DFD are shown in Figure 6.9.

Data dictionary for the DFD model of Example 6.2

```
move: integer /* number between 1 to 9 */
display: game+result
game: board
board: {integer}9
result: ["computer won", "human won", "drawn"]
```

EXAMPLE 6.3 (Supermarket Prize Scheme) A super market needs to develop a software that would help it to automate a scheme that it plans to introduce to encourage regular customers. In this scheme, a customer would have first register by supplying his/her residence address, telephone number, and the driving license number. Each customer who registers for this scheme is assigned a unique *customer number* (CN) by the computer. A customer can present his CN to the checkout staff when he makes any purchase. In this case, the value of his purchase is credited against his CN. At the end of each year, the supermarket intends to award surprise gifts to 10 customers who make the highest total purchase over the year. Also, it intends to award a 22 carat gold coin to every customer whose purchase exceeded ₹10,000. The entries against the CN are reset on the last day of every year after the prize winners' lists are generated.

The context diagram for the supermarket prize scheme problem of Example 6.3 is shown in [Figure 6.10](#). The level 1 DFD in [Figure 6.11](#). The level 2 DFD in [Figure 6.12](#).

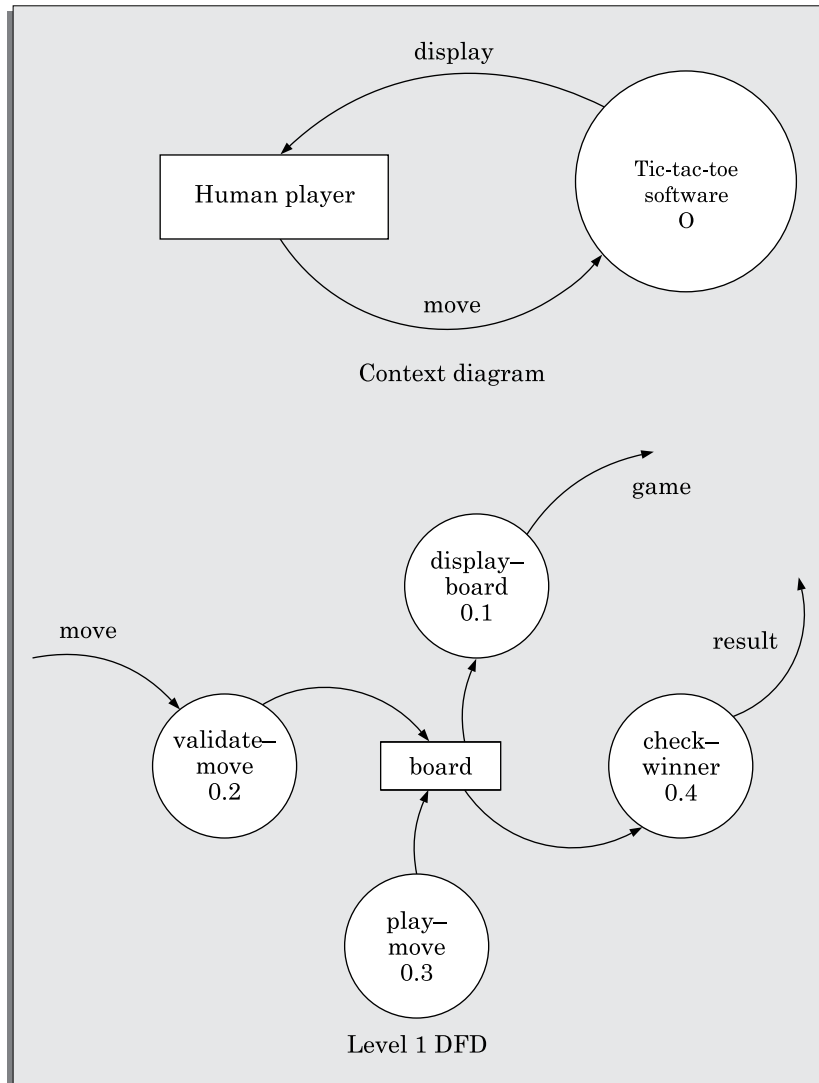


FIGURE 6.9 Context diagram and level 1 DFDs for Example 6.2.

Data dictionary for the DFD model of Example 6.3

address: name+house#+street#+city+pin

sales-details: {item+amount}* + CN

CN: integer

customer-data: {address+CN}*

sales-info: {sales-details}*

winner-list: surprise-gift-winner-list + gold-coin-winner-list

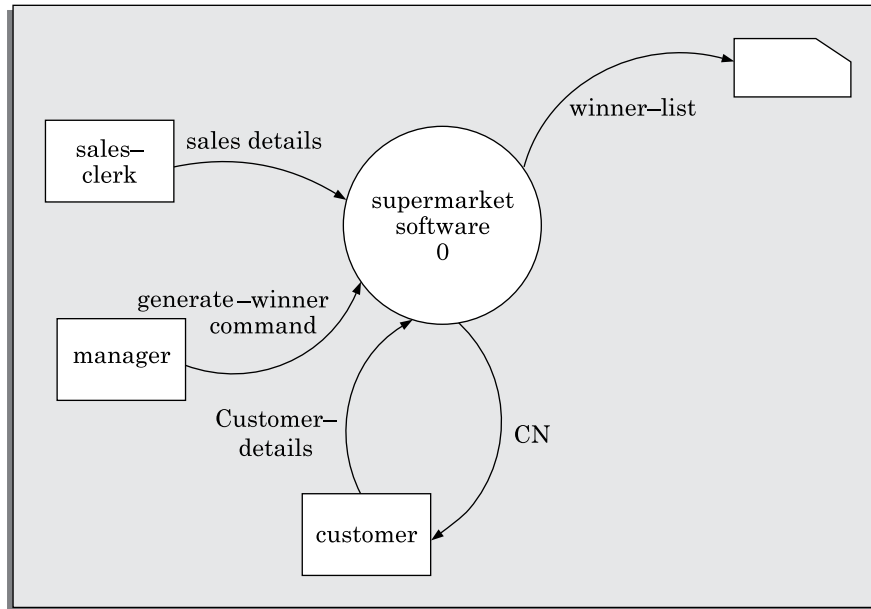


FIGURE 6.10 Context diagram for Example 6.3.

surprise-gift-winner-list: {address+CN}*

gold-coin-winner-list: {address+CN}*

gen-winner-command: command

total-sales: {CN+integer}^{*}

Observations: The following observations can be made from the Example 6.3.

1. The fact that the customer is issued a manually prepared customer identity card or that the customer hands over the identity card each time he makes a purchase has not been shown in the DFD. This is because these are item transfers occurring outside the computer.
2. The data `generate-winner-list` in a way represents control information (that is, command to the software) and no real data. We have included it in the DFD because it simplifies the structured design process as we shall realize after we practise solving a few problems. We could have also as well done without the `generate-winner-list` data, but this could have a bit complicated the design.
3. Observe in Figure 6.11 that we have two separate stores for the customer data and sales data. Should we have combined them into a single data store? The answer is—No, we should not. If we had combined them into a single data store, the structured design that would be carried out based on this model would become complicated. Customer data and sales data have very different characteristics. For example, customer data once created, does not change. On the other hand, the sales data changes frequently and also the sales data is reset at the end of a year, whereas the customer data is not.

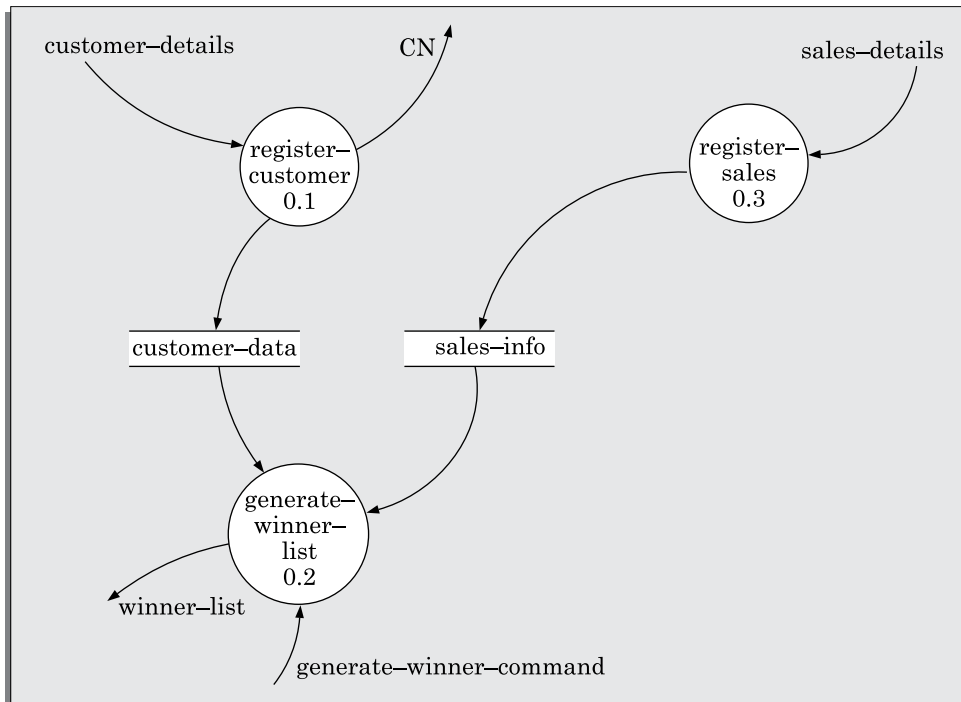


FIGURE 6.11 Level 1 diagram for Example 6.3.

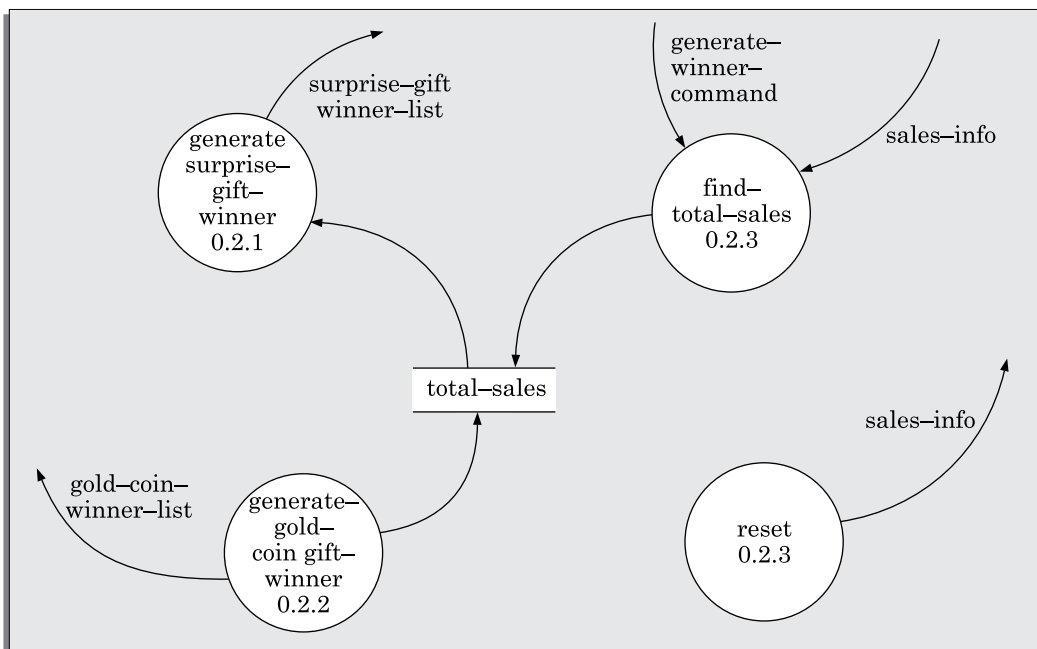


FIGURE 6.12 Level 2 diagram for Example 6.3.

EXAMPLE 6.4 [Trading-house Automation System (TAS)] A trading house wants us to develop a computerized system that would automate various book-keeping activities associated with its business. The following are the salient features of the system to be developed:

- The trading house has a set of regular customers. The customers place orders with it for various kinds of commodities. The trading house maintains the names and addresses of its regular customers. Each of these regular customers should be assigned a unique *customer identification number* (CIN) by the computer. The customers quote their CIN on every order they place.
- Once order is placed, as per current practice, the accounts department of the trading house first checks the credit-worthiness of the customer. The credit-worthiness of the customer is determined by analysing the history of his payments to different bills sent to him in the past. After automation, this task has to be done by the computer.
- If a customer is not credit-worthy, his orders are not processed any further and an appropriate order rejection message is generated for the customer.
- If a customer is credit-worthy, the items that he has ordered are checked against the list of items that the trading house deals with. The items in the order which the trading house does not deal with, are not processed any further and an appropriate apology message for the customer for these items is generated.
- The items in the customer's order that the trading house deals with are checked for availability in the inventory. If the items are available in the inventory in desired quantity, then:
 - A bill with the forwarding address of the customer is printed.
 - A material issue slip is printed. The customer can produce this material issue slip at the store house and take delivery of the items.
 - Inventory data is adjusted to reflect the sale to the customer.
- If any of the ordered items are not available in the inventory in sufficient quantity to satisfy the order, then these out-of-stock items along with the quantity ordered by the customer and the CIN are stored in a "pending-order" file for further processing to be carried out when the purchase department issues the "generate indent" command.
- The purchase department should be allowed to periodically issue commands to generate indents. When a command to generate indents is issued, the system should examine the "pending-order" file to determine the orders that are pending and determine the total quantity required for each of the items. It should find out the addresses of the vendors who supply these items by examining a file containing vendor details and then should print out indents to these vendors.
- The system should also answer managerial queries regarding the statistics of different items sold over any given period of time and the corresponding quantity sold and the price realised.

The context diagram for the trading house automation problem is shown in [Figure 6.13](#). The level 1 DFD in [Figure 6.14](#).

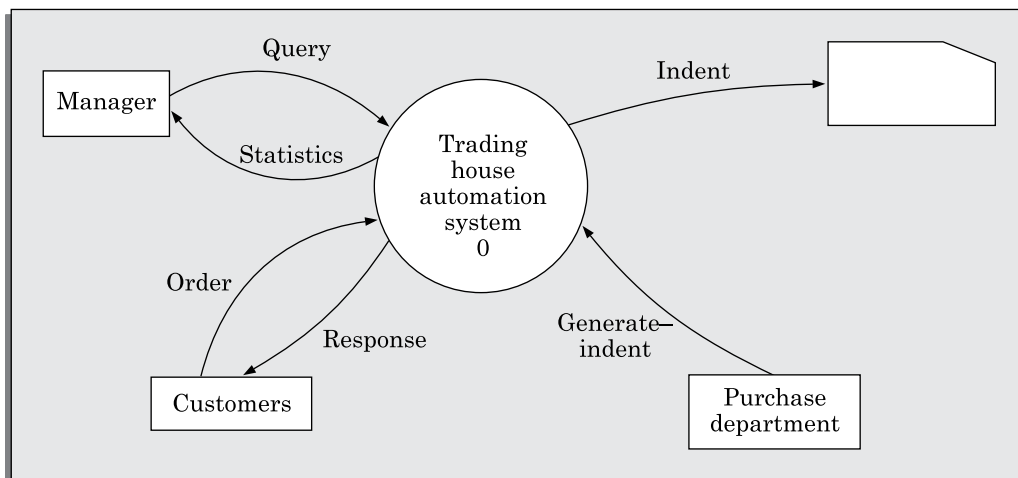


FIGURE 6.13 Context diagram for Example 6.4.

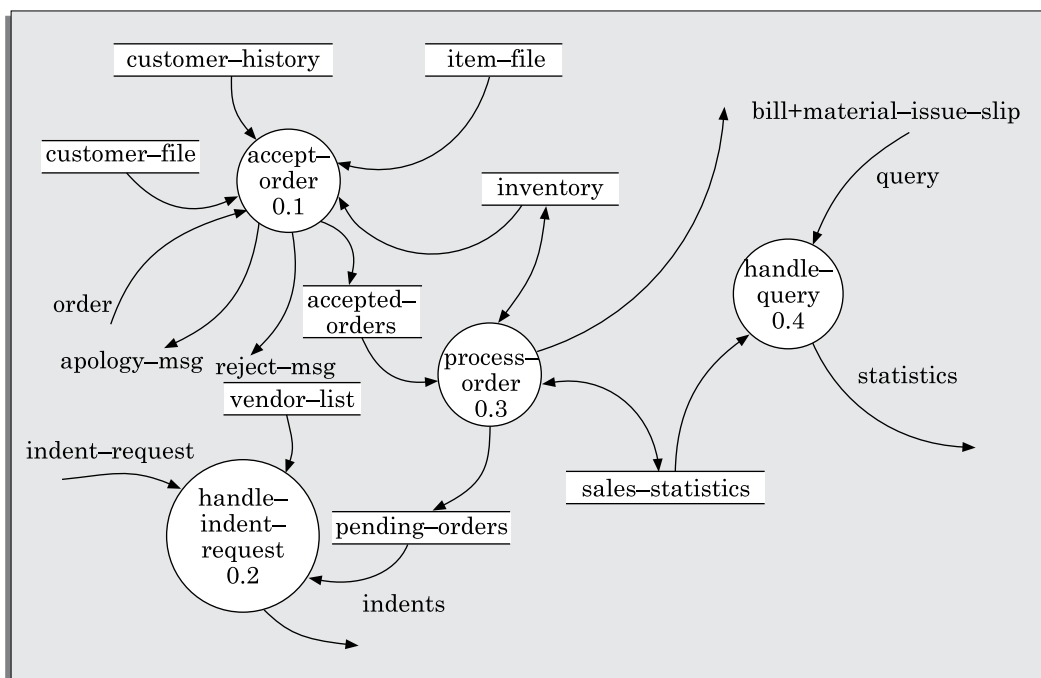


FIGURE 6.14 Level 1 DFD for Example 6.4.

Data dictionary for the DFD model of Example 6.4

response: [bill + material-issue-slip, reject-msg, apology-msg]

query: period /* query from manager regarding sales statistics*/

period: [date+date,month,year,day]

date: year + month + day
 year: integer
 month: integer
 day: integer
 customer-id: integer
 order: customer-id + {items + quantity}* + order#
 accepted-order: order /* ordered items available in inventory */
 reject-msg: order + message /* rejection message */
 pending-orders: customer-id + order# + {items+quantity}*
 customer-address: name+house#+street#+city+pin
 name: string
 house#: string
 street#: string
 city: string
 pin: integer
 customer-id: integer
 customer-file: {customer-address}* + customer-id
 bill: {item + quantity + price}* + total-amount + customer-address + order#
 material-issue-slip: message + item + quantity + customer-address
 message: string
 statistics: {item + quantity + price }*
 sales-statistics: {statistics}* + date
 quantity: integer
 order#: integer /* unique order number generated by the program */
 price: integer
 total-amount: integer
 generate-indent: command
 indent: {item+quantity}* + vendor-address
 indents: {indent}*
 vendor-address: customer-address
 vendor-list: {vendor-address}*
 item-file: {item}*
 item: string
 indent-request: command

Observations: The following observations can be made from Example 6.4.

1. In a DFD, if two data stores deal with different types of data, e.g. one type of data is invariant with time whereas another varies with time, (e.g. vendor address, and inventory data) it is a good idea to represent them as separate data stores.

The inventory data changes each time supply arrives and the inventory is updated or an item is sold, whereas the vendor data remains unchanged.

2. If we are developing the DFD model of a process which is already being manually carried out, then the names of the registers being maintained in the manual process would appear as data stores in the DFD model. For example, if TAS is currently

If two types of data always get updated at the same time, they should be stored in a single data store. Otherwise, separate data stores should be used for them.

being manually carried out, then normally there would registers corresponding to accepted orders, pending orders, vendor list, etc.

3. We can observe that DFDs enable a software developer to develop the data domain and functional domain model of the system at the same time. As the DFD is refined into greater levels of detail, the analyst performs an implicit functional decomposition. At the same time, the DFD refinement automatically results in refinement of corresponding data items.
4. The data that are maintained in physical registers in manual processing, become data stores in the DFD representation. Therefore, to determine which data should be represented as a data store, it is useful to try to imagine whether a set of data items would be maintained in a register in a manual system.

EXAMPLE 6.5 (Personal Library Software) Structured analysis for the personal library software is shown in Figure 6.15, Figure 6.16 and Figure 6.17.

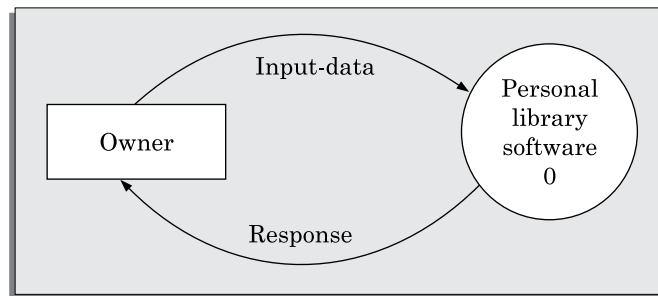


FIGURE 6.15 Context diagram for Example 6.5.

The level 1 DFD is shown in Figure 6.16.

The level 2 DFD for the manage OwnBook bubble is shown in [Figure 6.17](#).

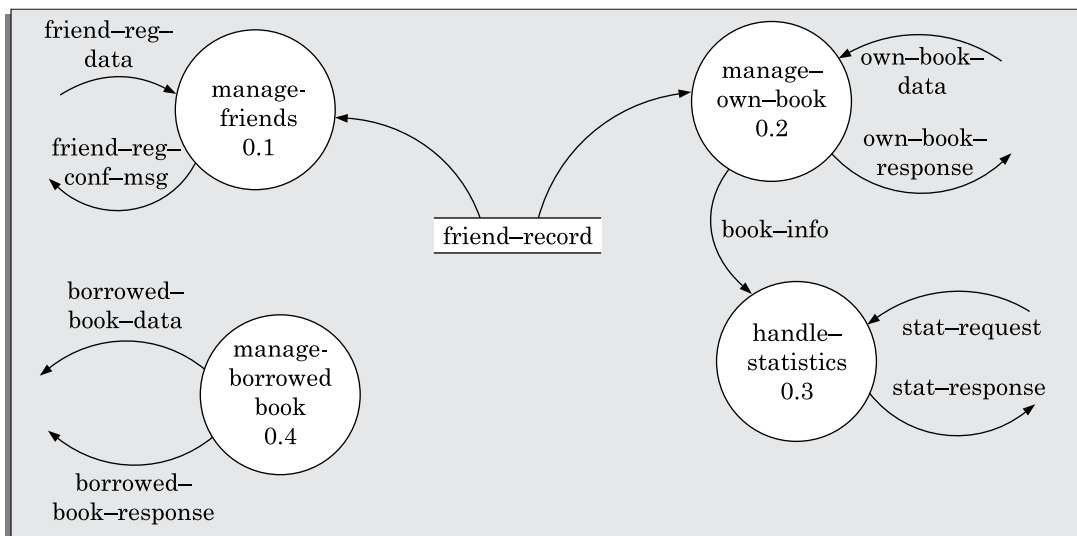


FIGURE 6.16 Level 1 DFD for Example 6.5.

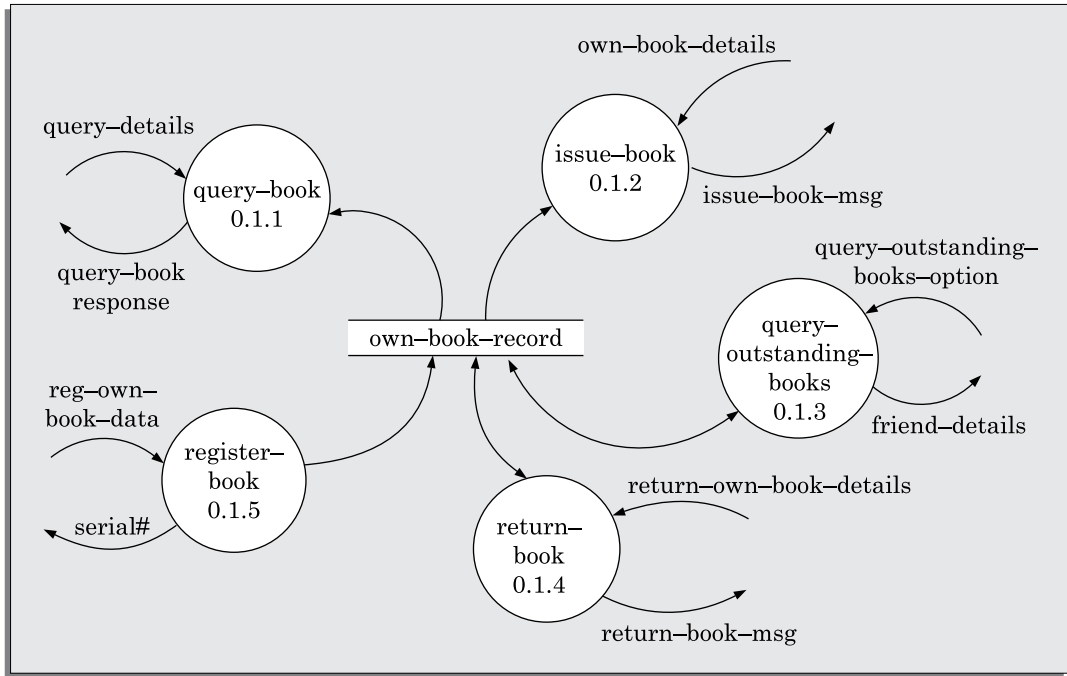


FIGURE 6.17 Level 2 DFD for Example 6.5.

Data dictionary for the DFD model of Example 6.5

input-data: friend-reg-data + own-book-data + stat-request + borrowed-book-data

response: friend-reg-conf-msg + own-book-response + stat-response +
borrowed-book-response

own-book-data: query-details + own-book-details + query-outstanding-books-option +
return-own-book-details + reg-own-book-data

own-book-response: query-book-response + issue-book-msg + friend-details + return-book-
msg + serial#.

borrowed-book-data: borrowed-book-details + book-return-details + display-books-option

borrowed-book-response: reg-msg + unreg-msg + borrowed-books-list

friend-reg-data: name + address + landline# + mobile#

own-book-details: friend-reg-data + book-title + data-of-issue

return-own-book-details: book-title + date-of-return

friend-details: name + address + landline# + mobile# + book-list

borrowed-book-details: book-title + borrow-date

serial#: integer

Observation: Observe that since there are more than seven functional requirements for the personal library software, related requirements have been combined to have only five bubbles in the level 1 diagram. Only level 2 DFD has been shown, since the other DFDs are trivial and need not be drawn.

Shortcomings of the DFD model

DFD models suffer from several shortcomings. The important shortcomings of DFD models are the following:

- Imprecise DFDs leave ample scope to be imprecise. In the DFD model, we judge the function performed by a bubble from its label. However, a short label may not capture the entire functionality of a bubble. For example, a bubble named find-book-position has only intuitive meaning and does not specify several things, e.g. what happens when some input information is missing or is incorrect. Further, the find-book-position bubble may not convey anything regarding what happens when the required book is missing.
- Not-well defined control aspects are not defined by a DFD. For instance, the order in which inputs are consumed and outputs are produced by a bubble is not specified. A DFD model does not specify the order in which the different bubbles are executed. Representation of such aspects is very important for modelling real-time systems.
- Decomposition: The method of carrying out decomposition to arrive at the successive levels and the ultimate level to which decomposition is carried out are highly subjective and depend on the choice and judgment of the analyst. Due to this reason, even for the same problem, several alternative DFD representations are possible. Further, many times it is not possible to say which DFD representation is superior or preferable to another one.
- Improper data flow diagram: The data flow diagramming technique does not provide any specific guidance as to how exactly to decompose a given function into its subfunctions and we have to use subjective judgment to carry out decomposition.

6.3.3 Extending DFD Technique to make it Applicable to Real-time Systems

In a real-time system, some of the high-level functions are associated with deadlines. Therefore, a function must not only produce correct results but also should produce them by some prespecified time. For real-time systems, execution time is an important consideration for arriving at a correct design. Therefore, explicit representation of control and event flow aspects are essential. One of the widely accepted techniques for extending the DFD technique to real-time system analysis is the Ward and Mellor technique [1985]. In the Ward and Mellor notation, a type of process that handles only control flows is introduced. These processes representing control processing are denoted using dashed bubbles. Control flows are shown using dashed lines/arrows.

Unlike Ward and Mellor, Hatley and Pirbhai [1987] show the dashed and solid representations on separate diagrams. To be able to separate the data processing and the control processing aspects, a *control flow diagram* (CFD) is defined. This reduces the complexity of the diagrams. In order to link the data processing and control processing diagrams, a notational reference (solid bar) to a control specification is used. The CSPEC describes the following:

- The effect of an external event or control signal.
- The processes that are invoked as a consequence of an event.

Control specifications represents the behaviour of the system in two different ways:

- It contains a *state transition diagram* (STD). The STD is a sequential specification of behaviour.
- It contains a *program activation table* (PAT). The PAT is a combinatorial specification of behaviour. PAT represents invocation sequence of bubbles in a DFD.

6.4 STRUCTURED DESIGN

The aim of structured design is to transform the results of the structured analysis (that is, the DFD model) into a structure chart. A structure chart represents the software architecture. The various modules making up the system, the module dependency (i.e., which module calls which other modules), and the parameters that are passed among the different modules. The structure chart representation can be easily implemented using some programming language. Since the main focus in a structure chart representation is on module structure of a software and the interaction among the different modules, the procedural aspects (e.g., how a particular functionality is achieved) are not represented.

The basic building blocks using which structure charts are designed are as following:

Rectangular boxes: A rectangular box represents a module. Usually, every rectangular box is annotated with the name of the module it represents.

Module invocation arrows: An arrow connecting two modules implies that during program execution control is passed from one module to the other in the direction of the connecting arrow. However, just by looking at the structure chart, we cannot say whether a modules calls another module just once or many times. Also, just by looking at the structure chart, we cannot tell the order in which the different modules are invoked.

Data flow arrows: These are small arrows appearing alongside the module invocation arrows. The data flow arrows are annotated with the corresponding data name. Data flow arrows represent the fact that the named data passes from one module to the other in the direction of the arrow.

Library modules: A library module is usually represented by a rectangle with double edges. Libraries comprise the frequently called *modules*. Usually, when a module is invoked by many other modules, it is made into a library module.

Selection: The diamond symbol represents the fact that one module of several modules connected with the diamond symbol is invoked depending on the outcome of the condition attached with the diamond symbol.

Repetition: A loop around the control flow arrows denotes that the respective modules are invoked repeatedly.

In any structure chart, there should be one and only one module at the top, called the *root*. There should be at most one control relationship between any two modules in the structure chart. This means that if module A invokes module B, module B cannot invoke module A. The main reason behind this restriction is that we can consider the different modules of a structure chart to be arranged in layers or levels. The principle of abstraction does not allow lower-level modules to be aware of the existence of the high-level modules.

However, it is possible for two higher-level modules to invoke the same lower-level module. An example of a properly layered design and another of a poorly layered design are shown in Figure 6.18.

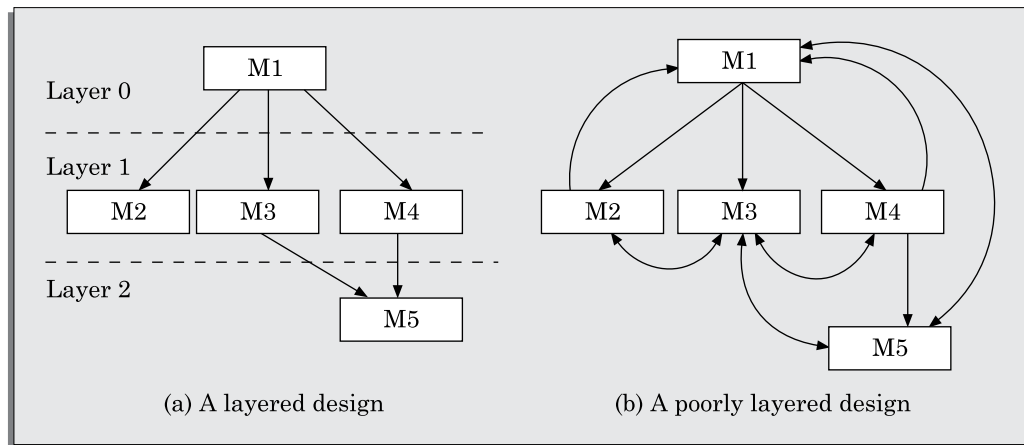


FIGURE 6.18 Examples of properly and poorly layered designs.

Flow chart versus structure chart

We are all familiar with the flow chart representation of a program. Flow chart is a convenient technique to represent the flow of control in a program. A structure chart differs from a flow chart in three principal ways:

- It is usually difficult to identify the different modules of a program from its flow chart representation.
- Data interchange among different modules is not represented in a flow chart.
- Sequential ordering of tasks that is inherent to a flow chart is suppressed in a structure chart.

6.4.1 Transformation of a DFD Model into Structure Chart

Systematic techniques are available to transform the DFD representation of a problem into a module structure represented by as a structure chart. Structured design provides two strategies to guide transformation of a DFD into a structure chart:

- Transform analysis
- Transaction analysis

At each level of transformation, it is important to first determine whether the transform or the transaction analysis is applicable to a particular DFD.

Normally, one would start with the level 1 DFD, transform it into module representation using either the transform or transaction analysis and then proceed toward the lower level DFDs.

Whether to apply transform or transaction processing?

Given a specific DFD of a model, how does one decide whether to apply transform analysis or transaction analysis? For this, one would have to examine the data input to the diagram. The data input to the diagram can be easily spotted because they are represented by dangling arrows. If all the data flow into the diagram are processed in similar ways (i.e., if all the input data flow arrows are incident on the same bubble in the DFD) then transform analysis is applicable. Otherwise, transaction analysis is applicable. Normally, transform analysis is applicable only to very simple processing.

Please recollect that the bubbles are decomposed until it represents a very simple processing that can be implemented using only a few lines of code. Therefore, transform analysis is normally applicable at the lower levels of a DFD model. Each different way in which data is processed corresponds to a separate transaction. Each transaction corresponds to a functionality that lets a user perform a meaningful piece of work using the software.

Transform analysis

Transform analysis identifies the primary functional components (modules) and the input and output data for these components. The first step in transform analysis is to divide the DFD into three types of parts:

- Input.
- Processing.
- Output.

The input portion in the DFD includes processes that transform input data from physical (e.g., character from terminal) to logical form (e.g., internal tables, lists, etc.). Each input portion is called an *afferent branch*.

The output portion of a DFD transforms output data from logical form to physical form. Each output portion is called an *efferent branch*. The remaining portion of a DFD is called *central transform*.

In the next step of transform analysis, the structure chart is derived by drawing one functional component each for the central transform, the afferent and efferent branches. These are drawn below a root module, which would invoke these modules.

Identifying the input and output parts requires experience and skill. One possible approach is to trace the input data until a bubble is found whose output data cannot be deduced from its inputs alone. Processes which validate input are not central transforms. Processes which sort input or filter data from it are central transforms. The first level of structure chart is produced by representing each input and output unit as a box and each central transform as a single box.

In the third step of transform analysis, the structure chart is refined by adding subfunctions required by each of the high-level functional components. Many levels of functional components may be added. This process of breaking functional components into subcomponents is called *factoring*. Factoring includes adding read and write modules, error-handling modules, initialisation and termination process, identifying consumer modules etc. The factoring process is continued until all bubbles in the DFD are represented in the structure chart.

PROBLEM 6.1 Draw the structure chart for the RMS software of Example 6.1.

Solution: By observing the level 1 DFD of Figure 6.8, we can identify validate-input as the afferent branch and write-output as the efferent branch. The remaining (i.e., compute-rms) as the central transform. By applying the step 2 and step 3 of transform analysis, we get the structure chart shown in Figure 6.19.

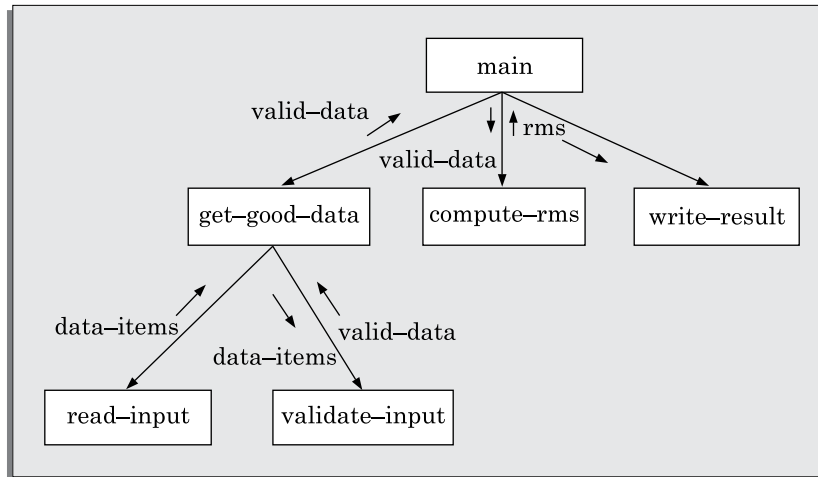


FIGURE 6.19 Structure chart for Problem 6.1.

PROBLEM 6.2 Draw the structure chart for the tic-tac-toe software of Example 6.2.

Solution: The structure chart for the Tic-tac-toe software is shown in Figure 6.20. Observe that the check-game-status bubble, though produces some outputs, is not really responsible for converting logical data to physical data. On the other hand, it carries out the processing involving checking game status. That is the main reason, why we have considered it as a central transform and not as an efferent type of module.

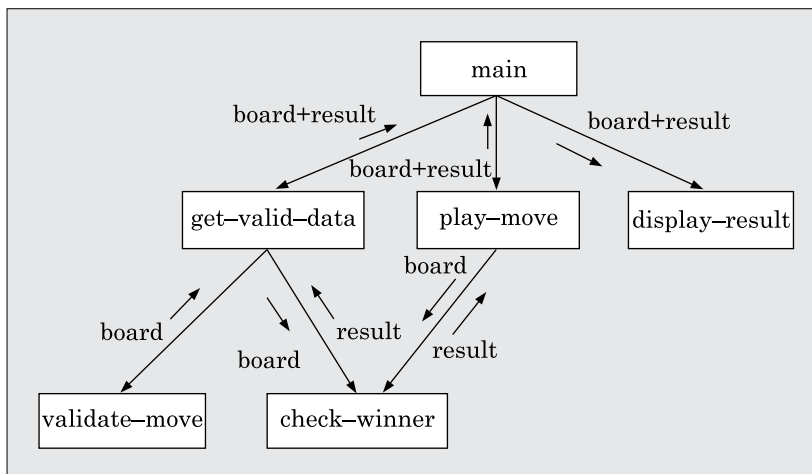


FIGURE 6.20 Structure chart for Problem 6.2.

Transaction analysis

Transaction analysis is an alternative to transform analysis and is useful while designing transaction processing programs. A transaction allows the user to perform some specific type of work by using the software. For example, 'issue book', 'return book', 'query book', etc., are transactions.

As in transform analysis, first all data entering into the DFD need to be identified. In a transaction-driven system, different data items may pass through different computation paths through the DFD. This is in contrast to a transform centered system where each data item entering the DFD goes through the same processing steps. Each different way in which input data is processed is a transaction. A simple way to identify a transaction is the following. Check the input data. The number of bubbles on which the input data to the DFD are incident defines the number of transactions. However, some transactions may not require any input data. These transactions can be identified based on the experience gained from solving a large number of examples.

For each identified transaction, trace the input data to the output. All the traversed bubbles belong to the transaction. These bubbles should be mapped to the same module on the structure chart. In the structure chart, draw a root module and below this module draw each identified transaction as a module. Every transaction carries a tag identifying its type. Transaction analysis uses this tag to divide the system into transaction modules and a transaction-center module.

PROBLEM 6.3 Draw the structure chart for the Supermarket Prize Scheme software of Example 6.3.

Solution: The structure chart for the Supermarket Prize Scheme software is shown in Figure 6.21.

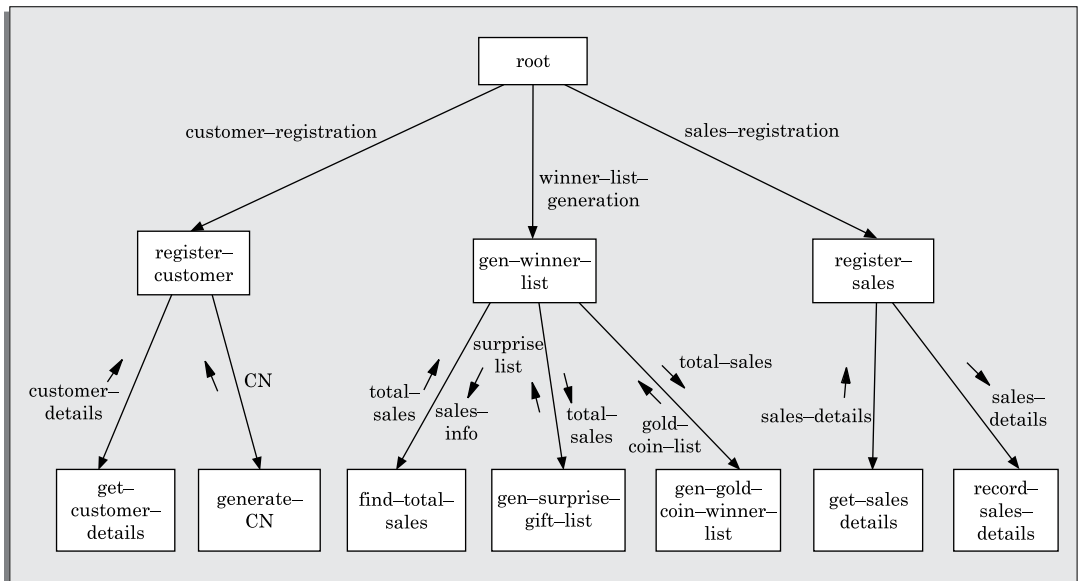


FIGURE 6.21 Structure chart for Problem 6.3.

PROBLEM 6.4 Draw the structure chart for the *trade-house automation system* (TAS) software of Example 6.4.

Solution: The structure chart for the *trade-house automation system* (TAS) software of Example 6.4 is shown in Figure 6.22.

By observing the level 1 DFD of Figure 6.14, we can see that the data input to the diagram are handled by different bubbles and therefore transaction analysis is applicable to this DFD. Input data to this DFD are handled in three different ways (accept-order, accept-indent-request, and handle-query), we have three different transactions corresponding to these as shown in Figure 6.22.

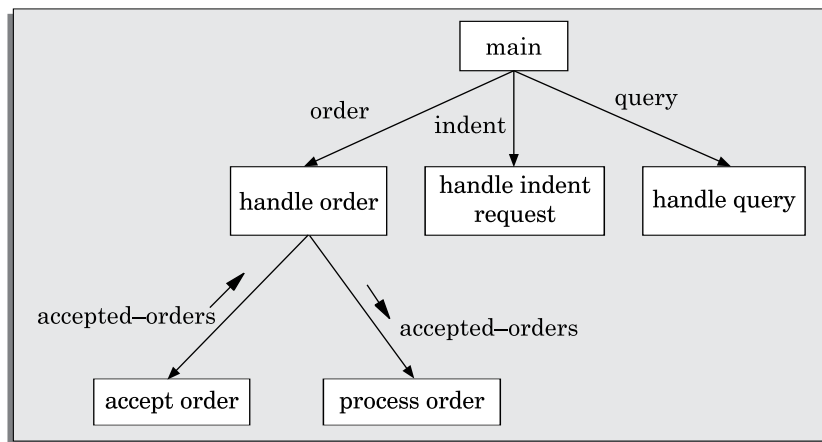


FIGURE 6.22 Structure chart for Problem 6.4.

Word of caution

We should view transform and transaction analyses as guidelines, rather than rules. We should apply these guidelines in the context of the problem and handle the pathogenic cases carefully.

PROBLEM 6.5 Draw the structure chart for the personal library software of Example 6.5.

Solution: The structure chart for the personal library software is shown in Figure 6.23.

6.5 DETAILED DESIGN

During detailed design the pseudo code description of the processing and the different data structures are designed for the different modules of the structure chart. These are usually described in the form of module specifications (MSPEC). MSPEC is usually written using structured English. The MSPEC for the non-leaf modules describe the different conditions under which the responsibilities are delegated to the lower-level modules. The MSPEC for the leaf-level modules should describe in algorithmic form how the primitive processing steps are carried out. To develop the MSPEC of a module, it is usually necessary to refer to the DFD model and the SRS document to determine the functionality of the module.

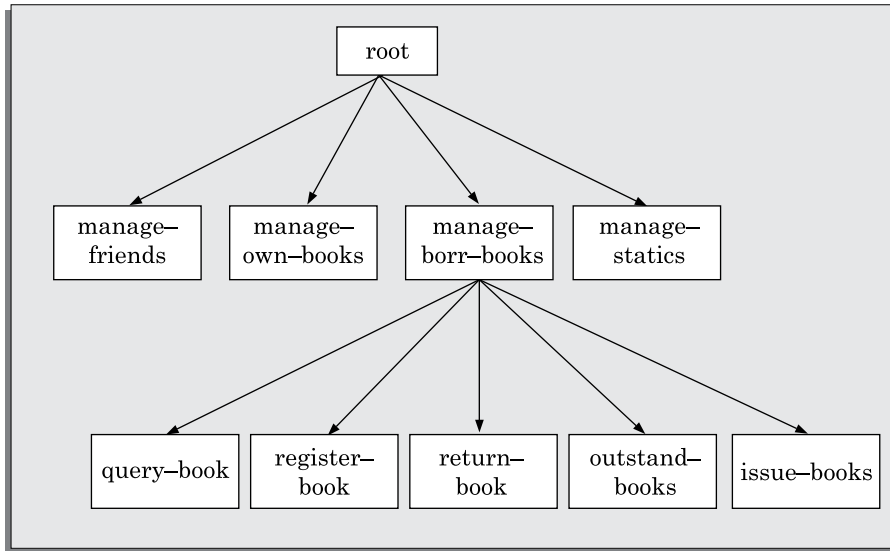


FIGURE 6.23 Structure chart for Problem 6.5.

6.6 DESIGN REVIEW

After a design is complete, the design is required to be reviewed. The review team usually consists of members with design, implementation, testing, and maintenance perspectives, who may or may not be the members of the development team. Normally, members of the team who would code the design, and test the code, the analysts, and the maintainers attend the review meeting. The review team checks the design documents especially for the following aspects:

Traceability: Whether each bubble of the DFD can be traced to some module in the structure chart and *vice versa*. They check whether each functional requirement in the SRS document can be traced to some bubble in the DFD model and *vice versa*.

Correctness: Whether all the algorithms and data structures of the detailed design are correct.

Maintainability: Whether the design can be easily maintained in future.

Implementation: Whether the design can be easily and efficiently be implemented.

After the points raised by the reviewers is addressed by the designers, the design document becomes ready for implementation.

SUMMARY

- In this chapter, we discussed a sample function-oriented software design methodology called structured analysis/structured design (SA/SD) which incorporates features of some important design methodologies.
- Methodologies like SA/SD give us a recipe for developing a good design according to the different goodness criteria we had discussed in Chapter 5.

- item SA/SD consists of two important parts—structured analysis and structured design.
- The goal of structured analysis is to perform a functional decomposition of the system. Results of structured analysis is represented using data flow diagrams (DFDs). The DFD representation is difficult to implement using a traditional programming language. The DFD representation can be systematically be transformed to structure chart representation. The structure chart representation can be easily implemented using a conventional programming language.
- During structured design, the DFD representation obtained during structured analysis is transformed into a structure chart representation.
- Several CASE tools are available to support the software design process carried out using the important function-oriented design methodologies. In addition to laying out the DFDs, structure charts, maintaining the data dictionary, and helping in traceability analysis, these CASE tools can also perform some elementary consistency checking, e.g., they can usually check whether a DFD is balanced or not.

EXERCISES



MULTIPLE CHOICE QUESTIONS

For each of the following questions, only one of the options is correct. Choose the correct option:

1. A data flow diagram represents:
 - (a) The conditions based on which a data may be processed
 - (b) The order in which different activities are carried out
 - (c) The transformation of data through processing stations
 - (d) The order in which various functions of a program are invoked
2. A DFD depicts which of the following:
 - (a) Flow of data
 - (b) Flow of control
 - (c) Flow of statements
 - (d) None of the above
3. Which of the following statements is not true of data flow diagrams (DFDs)?
 - (a) Hierarchical diagram.
 - (b) Represent code structure
 - (c) Do not represent decisions and control flows
 - (d) Represent functional decomposition
4. In a procedural design approach, during the detailed design stage, which of the following is undertaken?
 - (a) Module structure is designed
 - (b) Data flow representation is developed
 - (c) Data structures and algorithms for the individual modules are developed
 - (d) Structure chart is developed